

Minería de Textos. Práctica Bloque 2.

Carlos Sanchiz Pérez

Nicolás Vives Vicente

Escuela Politécnica Superior

Máster Universitario en Ciencia de Datos

Minería de Textos

Curso académico [2025-2026]



Universitat d'Alacant
Universidad de Alicante

Índice

1. Regresor logístico binario	3
1.1. División del conjunto en <i>train</i> , <i>validation</i> , <i>test</i>	3
1.2. Detener el entrenamiento según los resultados de evaluación	4
1.3. Gráfico de evolución de la función de pérdida	6
2. Regresor logístico multinomial	7
3. Skip-grams: Parte 1	9
4. Skip-grams: Parte 2	12
5. Modelos de lengua basados en transformers	15
5.1. Modificación de funciones	16
5.2. Estudio	20

1. Regresor logístico binario

Enunciado: Modifica el código para que el conjunto de entrenamiento se divida en entrenamiento, validación y test. Usa el conjunto de evaluación para determinar cuándo detener el entrenamiento. Añade un gráfico que muestre la evolución de la función de pérdida en el conjunto de entrenamiento y en el de validación.

1.1. División del conjunto en *train*, *validation*, *test*

Comenzamos modificando el código para que el conjunto de entrenamiento se divida en entrenamiento, validación y test.

```
indices = torch.arange(X.shape[0]).to(device) # tensor con los números
        del 0 al 99 (o el tamaño que tenga X)

# máscara de test (20%) 1 de cada 5 elementos
mask_test = (indices % 5 == 0)

# máscara de validación (20%) el siguiente 1 de cada 5 elementos
mask_val = (indices % 5 == 1)

# máscara de entrenamiento (60%)
mask_train = ~(mask_test | mask_val)

X_train, y_train = X[mask_train], y[mask_train]
X_val, y_val = X[mask_val], y[mask_val]
X_test, y_test = X[mask_test], y[mask_test]

print(f"X_train.shape = {X_train.shape}")
print(f"y_train.shape = {y_train.shape}")
print(f"X_val.shape = {X_val.shape}")
print(f"y_val.shape = {y_val.shape}")
print(f"X_test.shape = {X_test.shape}")
print(f"y_test.shape = {y_test.shape}")
```

Listing 1: División del conjunto de entrenamiento en entrenamiento, validación y test.

Consideramos que la mejor división en este caso era 60% *train*, 20% *validation* y 20% *test*. En lugar de seguir el patrón de ejemplo que se propone en el *notebook*, al tener que dividir en tres conjuntos, hemos optado por utilizar la operación matemática de módulo para crear las máscaras. Primero creamos un tensor con el mismo tamaño que *X*, que comience en 0 y sume uno en cada posición. Para crear las máscaras, creamos un tensor

con los restos de cada número al dividir entre 5 (módulo 5) dando lugar a 5 valores posibles (0, 1, 2, 3, 4). Ahora ya se pueden hacer dos divisiones del 20 % para *validation* y *test*. Se consideran *True* todas las posiciones en las que haya un 0 para *test* y *True* todas las posiciones en las que haya un 1 para *validation*, dejando en cada máscara el resto en *False*. Ahora el opuesto de la unión de estas dos máscaras será la máscara de *train*. Si se quisiera evitar cualquier tipo de problema se debería aplicar además un *shuffle* que introduzca aleatoriedad, ya que si los datos originales tuvieran algún tipo de orden, el uso del módulo podría no garantizar un buen muestreo.

1.2. Detener el entrenamiento según los resultados de evaluación

Actualizamos la función de entrenamiento para que el conjunto de validación determine cuándo se detiene el entrenamiento.

```
def train_with_early_stopping(X_train, y_train, X_val, y_val, weights,
                              bias, lr=0.01, training_steps=1000, valid_steps=10, patience=5):
    best_val_loss = float('inf')
    patience_counter = 0 # contador de pasos sin mejorar
    best_weights = weights.clone()
    best_bias = bias.clone()

    history_steps = []
    history_train_loss = []
    history_val_loss = []

    for i in range(training_steps):
        y_pred = forward(X_train, weights, bias)

        # quitamos la actualización de los pesos para poder comparar
        # exactamente en la misma etapa los errores de entrenamiento y validación

        if i % valid_steps == 0:
            train_loss = binary_cross_entropy(y_train, y_pred).item()

            y_pred_val = forward(X_val, weights, bias)
            val_loss = binary_cross_entropy(y_val, y_pred_val).item()

            history_steps.append(i)
            history_train_loss.append(train_loss)
            history_val_loss.append(val_loss)
```

```
print(f'Step [{i}/{training_steps}], Train Loss: {train_loss:.4f}, Val Loss: {val_loss:.4f}')

if val_loss < best_val_loss:
    best_val_loss = val_loss
    patience_counter = 0
    best_weights = weights.clone()
    best_bias = bias.clone()
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f'\nEarly stopping en el paso {i}!')
        break

# después de la evaluación actualizamos los pesos
grad_w, grad_b = loss_backward(y_train, y_pred, X_train)
weights, bias = optimizer_step(weights, bias, grad_w, grad_b, lr
)

return best_weights, best_bias, best_val_loss, history_steps,
history_train_loss, history_val_loss
```

Listing 2: Actualización de la función de entrenamiento con *early stopping* por validación.

Para añadir a la función de entrenamiento la condición de parada según los resultados de validación, se requiere que la entrada separe entre los conjuntos de entrenamiento y validación. Comenzamos inicializando la mejor pérdida en validación como infinito y un contador de paciencia en 0 para indicar el número de bucles de validación que estamos dispuestos a esperar que el modelo no mejore antes de detener el entrenamiento. Añadimos tres listas vacías antes del bucle de entrenamiento para almacenar los errores.

Para poder comparar los errores en el mismo paso exactamente hemos realizado la predicción y movido la actualización de pesos al final. Antes de esto, cada `valid_steps` número de pasos se calcula la pérdida en ambos conjuntos para compararlas y se van completando las listas con el paso y las pérdidas calculadas. Si la pérdida de validación es menor que la mejor pérdida hasta ahora, se actualiza esta y se guardan los mejores pesos del modelo. En caso contrario se suma un bucle de paciencia. Por último, ahora sí se actualizan los pesos para comenzar con la siguiente iteración del bucle.

1.3. Gráfico de evolución de la función de pérdida

Para añadir un gráfico que muestre la evolución de la función de pérdida en el conjunto de entrenamiento y en el de validación debemos modificar la función anterior para ir guardando las pérdidas en lugar de solo imprimirlas, por ello, solo mostramos las nuevas líneas de código añadidas.

Ahora simplemente dibujamos las líneas mediante los valores almacenados en las listas.

```
plt.figure(figsize=(10, 6))

plt.plot(history_steps, history_train_loss, label='Train Loss', color='blue', linewidth=2, marker='o', markersize=4)
plt.plot(history_steps, history_val_loss, label='Validation Loss', color='red', linewidth=2, marker='s', markersize=4)

plt.title('Evolución de la Función de Pérdida durante el Entrenamiento',
          fontsize=14)
plt.xlabel('Pasos de Entrenamiento (Steps)', fontsize=12)
plt.ylabel('Pérdida (Binary Cross-Entropy)', fontsize=12)
plt.legend(fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Listing 3: Incorporación de listas para guardar los pasos y los errores de cada conjunto.

Con matplotlib dibujamos el gráfico de evolución de las funciones de pérdida, en el eje x el número de pasos y en el eje y el término del error. Se puede observar en azul la pérdida en el conjunto de entrenamiento y en rojo la pérdida en validación.

Podemos observar en la Figura 1 cómo la pérdida en ambos conjuntos se reduce rápidamente. A partir de los 100 pasos aproximadamente, la pérdida en validación es ligeramente más baja que en el entrenamiento, lo que nos indica una ausencia de *overfitting*. En general es esperable que la pérdida en el conjunto de entrenamiento sea siempre algo más baja que en validación, ya que son los datos que el modelo ha visto. En este caso sucede lo contrario, lo que puede deberse a que al ser un conjunto de datos tan reducido, de solo 100 muestras (60 para entrenamiento), las 20 muestras de validación sean algo más sencillas de clasificar que las de entrenamiento, dando lugar en todo momento a una pérdida más baja en el conjunto de datos no vistos.

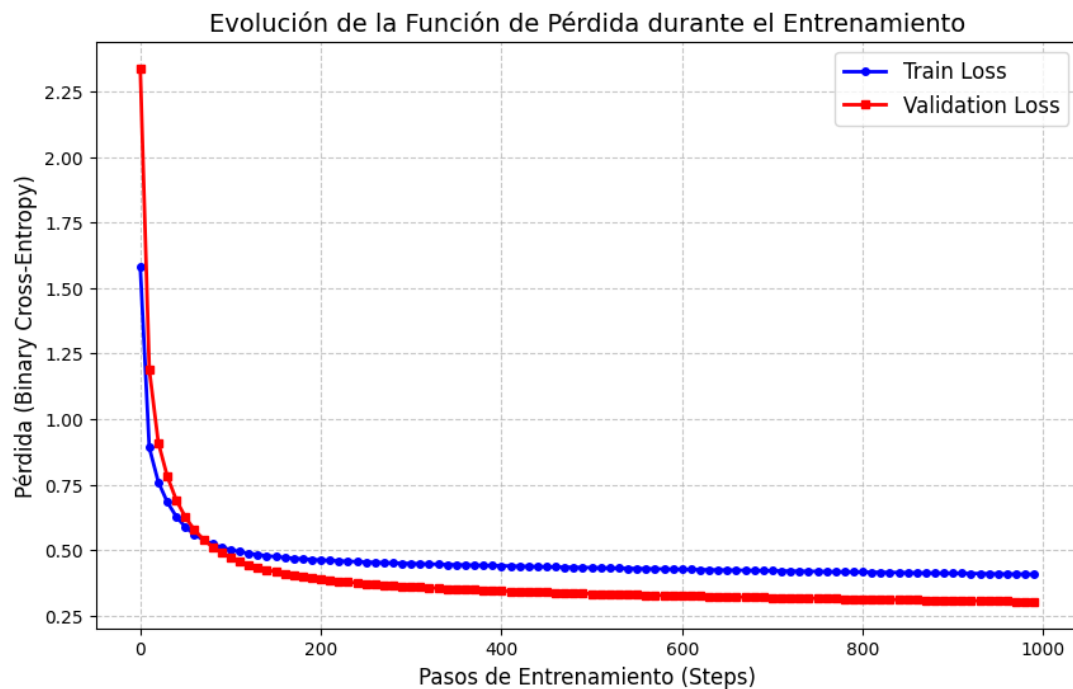


Figura 1: Evolución de la función de pérdida durante el entrenamiento en los conjuntos de entrenamiento y validación.

2. Regresor logístico multinomial

Enunciado: Estudia las probabilidades emitidas por el modelo ya entrenado para cada clase tanto con las frases del cuaderno como con algunas frases nuevas que tú propongamos. Juega con frases que estén a medio camino entre diferentes temáticas.

Comenzamos preparando el siguiente código para estudiar las probabilidades del modelo:

```
embeddings = model_embedding.encode(sentences_enunciado,
    convert_to_tensor=True).clone().detach()
embeddings = embeddings.to(device)
embeddings = embeddings.float()

with torch.no_grad():
    outputs = model(embeddings)
    probs = torch.exp(outputs)
```

Listing 4: Preparación del código para analizar las frases propuestas en el enunciado.

En la Tabla 1 se pueden observar los resultados obtenidos con el código anterior para

diferentes frases.

	Prob Política	Prob Deporte	Prob Cultura	Predicción
The election results were surprising	0.6133	0.1851	0.2016	Política
The new policy on healthcare was announced	0.7007	0.1633	0.1360	Política
The government passed a new education reform	0.7236	0.1485	0.1279	Política
The soccer team won the championship	0.1810	0.6372	0.1818	Deporte
The Olympics will be held next year	0.1671	0.6405	0.1924	Deporte
She won the gold medal in swimming	0.1537	0.6712	0.1751	Deporte
The museum exhibited contemporary art	0.1592	0.1647	0.6761	Cultura
The film festival was a huge success	0.1573	0.2039	0.6388	Cultura
The concert was sold out	0.1454	0.1826	0.6720	Cultura
A new law was proposed	0.5183	0.2686	0.2131	Política
The basketball game was exciting	0.2817	0.4192	0.2991	Deporte
The opera was a beautiful experience	0.2299	0.2584	0.5118	Cultura

Tabla 1: Probabilidades de las frases del enunciado.

Respecto a las frases del conjunto de entrenamiento, vemos como el modelo tiene bastante claro a que grupo pertenecen, siendo en cualquiera de ellas la probabilidad mayor siempre superior a 0.60. Por otro lado, un resultado similar conseguimos con las frases del conjunto test, siendo su probabilidad, en el grupo al que pertenece, superior al 0.50.

Vamos a proponer nuevas frases con la siguiente idea.

- Las tres primeras van a ser frases simples dónde el modelo no debería de tener problema a la hora de clasificar.
- Después, vamos a crear tres frases uniendo grupos de dos en dos, es decir, la primera combinación será una frase de política y deporte, la segunda de deporte y cultura, y por último, una de cultura y política. Además, también propondremos una combinación de los tres grupos.
- Para poner a prueba el modelo, propondremos también una frase que no tenga nada que ver con el resto y tres frases que en principio no deberían costarle pero que estarán en español.

En la Tabla 2 se pueden observar los resultados obtenidos.

- Para la propuesta de tres frases nuevas fáciles, vemos como el modelo distingue acertadamente las 3 veces, otorgando una probabilidad superior al 0.45 en todos los casos, al grupo correspondiente.
- Siguiendo con las frases 4, 5 y 6, que son las combinadas, vemos como el modelo otorga una mayor probabilidad a los grupos que se pretendía.

	P. Política	P. Deporte	P. Cultura	Predicción
The new law destroyed the country	0.5265	0.2370	0.2366	Política
Messi is the best football player	0.2377	0.5257	0.2366	Deporte
What do you prefer film or book?	0.2591	0.2777	0.4632	Cultura
The new president has a football club	0.3843	0.3696	0.2461	Política
My sister did a film about basketball	0.2155	0.4559	0.3286	Deporte
The government made a new law to encourage going to the cinema	0.3281	0.2332	0.4387	Cultura
The new president made a film about football	0.3531	0.3191	0.3278	Política
I like cake	0.3257	0.3436	0.3308	Deporte
No me considero ni de izquierdas ni de derechas	0.3481	0.3753	0.2766	Deporte
Me gusta cualquier deporte	0.2872	0.4311	0.2817	Deporte
De mayor quiero hacer una obra de teatro	0.2619	0.2919	0.4462	Cultura

Tabla 2: Probabilidades de frases creadas.

- Por otro lado, tanto la frase 7, como la 8, presentan unas probabilidades muy parecidas y una predicción indecisa, siendo todos los valores superiores al 0.30. Aunque pueden parecer resultados similares, se deben a razones opuestas, primero, en la frase número 7, el modelo capta relaciones con los tres temas, por lo que no es capaz de decidirse. Después, en la frase 8, que es una frase que no pertenece a ningún grupo, el modelo tampoco es capaz de decidirse, pero esta vez debido a que no encuentra relación con ninguno de ellos.
- Para las últimas frases, vemos como la frase 10 y 11 se predicen con una seguridad suficiente, mientras que en la frase número 9, se equivoca, prediciendo la frase como si fuera del tema deporte, mientras que es del tema política. Aunque ambos grupos tienen una probabilidad destacable, la del deporte es un poco superior.

3. Skip-grams: Parte 1

Enunciado: Modifica el código para que se pueda seleccionar el tamaño de la ventana L y realiza un pequeño estudio sobre cómo esto afecta a los *embeddings* obtenidos.

Modificamos la parte del código que se encuentra donde creamos las muestras positivas y negativas de la siguiente manera:

```
L = 1
skip_grams_positive = []
for i in range(L, len(corpus) - L):
    target = word_index[corpus[i]]
    context = []
    for j in range(1, L + 1):
        context.append(word_index[corpus[i - j]])
```

```

        context.append(word_index[corpus[i + j]])
    for c in context:
        skip_grams_positive.append([target, c])

skip_grams_negative = []
for i in range(len(corpus)):
    target = word_index[corpus[i]]
    context = [word_index[i] for i in random.sample(word_list, L*k)]
    for c in context:
        skip_grams_negative.append([target, c])

```

Listing 5: Código muestras positivas y negativas modificado

Ahora se compara la similitud de cada palabra con el resto. Una vez recogidos todos los datos, vamos a hacer la media de la similitud entre todas las palabras de dos grupos:

```

cos = nn.CosineSimilarity(dim=0)

gato = model.W(torch.tensor(word_index['gato']))
perro = model.W(torch.tensor(word_index['perro']))
pajaro = model.W(torch.tensor(word_index['pájaro']))
raton = model.W(torch.tensor(word_index['ratón']))
animales = [gato, perro, pajaro, raton]

verde = model.W(torch.tensor(word_index['verde']))
amarillo = model.W(torch.tensor(word_index['amarillo']))
rojo = model.W(torch.tensor(word_index['rojo']))
azul = model.W(torch.tensor(word_index['azul']))
colores = [verde, amarillo, rojo, azul]

guitarra = model.W(torch.tensor(word_index['guitarra']))
piano = model.W(torch.tensor(word_index['piano']))
violin = model.W(torch.tensor(word_index['violín']))
flauta = model.W(torch.tensor(word_index['flauta']))
instrumentos = [guitarra, piano, violin, flauta]

naranja = model.W(torch.tensor(word_index['naranja']))
pera = model.W(torch.tensor(word_index['pera']))
uva = model.W(torch.tensor(word_index['uva']))
manzana = model.W(torch.tensor(word_index['manzana']))
frutas = [naranja, pera, uva, manzana]

def similitud_media_de_dos_grupos(grupo1, grupo2):
    similitud = []

```

```

for i in grupo1:
    for j in grupo2:
        if i is not j:
            similitud.append(cos(i, j).item())
return np.mean(similitud)

grupos = [animales, colores, instrumentos, frutas]
nombres = ["animales", "colores", "instrumentos", "frutas"]

for i in range(len(grupos)):
    for j in range(i, len(grupos)):
        print(f"La similitud media entre {nombres[i]} y {nombres[j]} es: {
            similitud_media_de_dos_grupos(grupos[i], grupos[j]):.3f}")

```

Listing 6: Código de comparación de las palabras de cada grupo.

Podemos ver los resultados, con $L = 1$, en la siguiente tabla:

	Animales	Colores	Instrumentos	Frutas
Animales	0.996	0.376	0.846	0.073
Colores	0.376	0.993	0.799	0.948
Instrumentos	0.846	0.799	0.985	0.580
Frutas	0.073	0.948	0.580	0.999

Tabla 3: Similitud media entre grupos, según $L = 1$.

Vemos como las palabras de un mismo grupo comparten una similitud superior a 0.985 (La diagonal), mientras que cuando comparamos palabras de distintos grupos vemos como la media es bastante inferior. Para ver la influencia del parámetro L , vamos a calcular la media de la similitud de palabras que pertenecen a un mismo grupo, frente a la similitud de palabras de distintos grupos. Vamos a ver la evolución desde $L = 1$ hasta $L = 5$:

	Similitud (Mismo grupo)	Similitud (Distinto grupo)
$L = 1$	0.993	0.604
$L = 2$	0.942	0.604
$L = 3$	0.933	0.616
$L = 4$	0.937	0.713
$L = 5$	0.949	0.795

Tabla 4: Evolución de la similitud según L .

Viendo primero la evolución de palabras del mismo grupo vemos como, cuándo $L = 1$ se obtiene el mejor resultado, alcanzando una similitud de 0.99, de media. A medida que aumenta L , la similitud entre las palabras de un mismo grupo disminuye debido a que cada palabra tiene más parejas positivas, por lo que su embedding se ajusta a una distribución más amplia. La caída de 5 décimas de similitud entre $L = 1$ y $L = 2$, puede deberse a la presencia de “.” en el corpus, ya que hace de barrera entre palabras de distintos grupos, en $L = 1$.

Por otro lado, vemos como a medida que aumentamos L en la segunda columna, aumenta la similitud. Esto se debe a que cuándo aumenta L cada palabra adquiere más contexto, por ejemplo, en $L = 5$, cada palabra se relaciona con palabras de otros grupos, por lo que es de esperar que la similitud aumente.

4. Skip-grams: Parte 2

Enunciado: Sustituye la parte del código que usa la notación de Einstein por una multiplicación convencional de matrices seguida de una operación que se quede con los valores que nos interesan. Compara los tiempos de ejecución de ambas implementaciones.

Comenzamos modificando el modelo proporcionado cambiando la línea de cálculo de la suma de Einstein por la multiplicación de matrices y la posterior toma de la diagonal:

```
class Word2Vec2(nn.Module):
    def __init__(self, voc_size, embedding_size):
        super().__init__()
        self.W = nn.Embedding(voc_size, embedding_size) # target matrix
        self.C = nn.Embedding(voc_size, embedding_size) # context
        matrix

    def forward(self, Xt, Xc):
        w = self.W(Xt)
        c = self.C(Xc)
        dot = torch.mm(w, c.t()).diag()

        output_layer = F.sigmoid(dot)
        return output_layer
```

Listing 7: Modificación de Word2Vec para utilizar una multiplicación convencional de matrices seguida de una toma de la diagonal.

Basta modificar una línea de la clase en la que se define cómo se realiza el producto.

Una vez definido el modelo comparamos los tiempos de entrenamiento entre ambos siendo lo más fieles posibles al código original del *notebook* y eliminando únicamente los prints.

Los resultados obtenidos para 2500 pasos de entrenamiento fueron en promedio:

- **torch.einsum**: 3.8932 segundos
- **torch.mm(w, c.t()).diag()**: 2.6930 segundos

Podemos ver como a diferencia de lo que esperábamos, resulta más lento el uso de la suma de Einstein que la multiplicación de matrices. La diferencia en el entrenamiento completo es pequeña porque el cálculo del producto escalar representa un porcentaje ínfimo del tiempo de cada paso. La inmensa mayoría del tiempo se consume generando muestras aleatorias y calculando los gradientes, así que decidimos realizar una prueba aislada únicamente para el cálculo de las operaciones y así evaluar la diferencia real entre ambas.

Tras hacer esta prueba obtuvimos los siguientes resultados:

- **torch.einsum**: 0.3537 segundos
- **torch.mm(w, c.t()).diag()**: 0.2211 segundos

Aislando el efecto del cálculo, confirmamos que para este tamaño de *batch* es más rápido el producto de matrices tomando la diagonal que la suma de Einstein. Esto nos motivó a realizar una prueba variando el tamaño del *batch* para así poder evaluar su impacto en ambas operaciones:

```
embedding_size = 2
batch_sizes = [32, 64, 128, 256, 512, 1024, 2048, 4096]
iteraciones = 500

device = 'cuda' if torch.cuda.is_available() else 'cpu'

tiempos_einsum = []
tiempos_matmul = []

for b_size in batch_sizes:

    w = torch.randn(b_size, embedding_size, device=device)
    c = torch.randn(b_size, embedding_size, device=device)
```

```

# warmup
for _ in range(10):
    _ = torch.einsum('ij,ji->i', w, c.t())
    _ = torch.mm(w, c.t()).diag()

if device == 'cuda': torch.cuda.synchronize()

# medir Einsum
start = time.time()
for _ in range(iteraciones):
    _ = torch.einsum('ij,ji->i', w, c.t())
if device == 'cuda': torch.cuda.synchronize()
tiempos_einsum.append(time.time() - start)

# medir Matmul
start = time.time()
for _ in range(iteraciones):
    _ = torch.mm(w, c.t()).diag()
if device == 'cuda': torch.cuda.synchronize()
tiempos_matmul.append(time.time() - start)

```

Listing 8: Comparación de los diferentes tiempos de ejecución variando el tamaño del batch

Si usamos PyTorch en GPU debemos medir el tiempo asíncrono y además realizamos un *warmup* ya que hemos visto en pruebas de rendimiento rigurosas que es conveniente realizar un par de pasadas en vacío antes de empezar a medir el tiempo para inicializar la memoria y despertar a la GPU.

En la Figura 2 podemos observar cómo para los 3 primeros tamaños de batch (32, 64 y 128) el cálculo es más rápido mediante la multiplicación de matrices que por la multiplicación de Einstein. Según la información que hemos podido encontrar, esto se debe a que `torch.einsum` tiene una mayor sobrecarga inicial, el intérprete necesita leer el *string*, analizar las dimensiones y planificar cuál es la estrategia óptima para el cálculo. Esto provoca que para matrices más pequeñas el cálculo de la suma de Einstein sea más lento que hacer el cálculo completo del producto de matrices y descartar los valores de fuera de la diagonal que no nos interesan.

A medida que el tamaño del *batch* crece, concretamente a partir de 256, esto se invierte, creciendo apenas un poco el tiempo de cálculo mediante la suma de Einstein y multiplicándose por más de 1000 el tiempo para la multiplicación de matrices de tamaño 4096x4096. Esto se debe a que la sobrecarga inicial de la suma de Einstein es siempre la misma, pero con un *batch* de 4096 para la multiplicación de matrices, se calculan casi 17

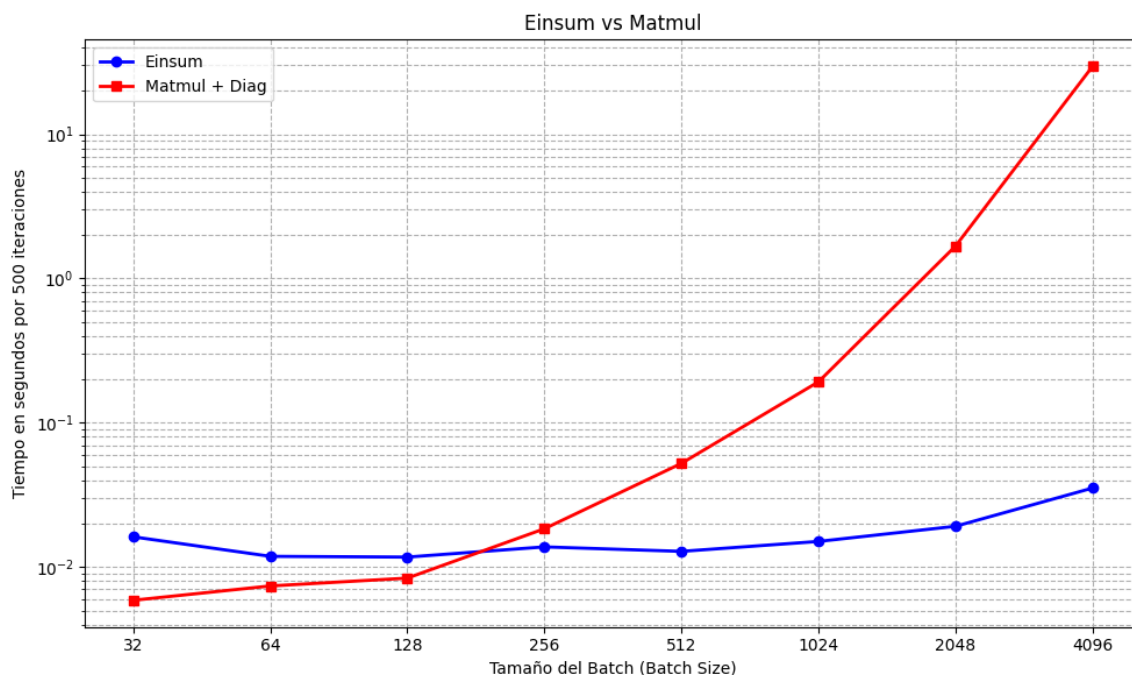


Figura 2: Comparativa del tiempo de ejecución entre einsum y la multiplicación matricial en función del tamaño del batch.

millones de valores. Ocupar esa memoria, y luego prescindir de todo menos 4096 valores es muy ineficiente.

5. Modelos de lengua basados en transformers

Enunciado: Modifica el código del modelo de lengua basado en transformers para que use *multi-query attention* sin implementar todavía una caché KV. Realiza un pequeño estudio sobre cómo afecta esto a la calidad del modelo; para esto puedes medir qué probabilidad da el modelo a algunas frases similares a las del conjunto de entrenamiento. Adicionalmente, estudia cómo afecta a la calidad del modelo el uso de una caché KV que tendrás que implementar. Aunque sería deseable poder medir el impacto de ambas cosas en los tiempos de ejecución, no es necesario que lo hagas, ya que probablemente no puedas medirlo con precisión suficiente salvo que incrementes el tamaño de los datos de entrenamiento y el número de parámetros del modelo. Explica en tu respuesta las ideas básicas tanto de *multi-query attention* como de la caché KV.

5.1. Modificación de funciones

En primer lugar, vamos a implementar una función que use *multi-query attention*. Nos vamos a basar en la función *MultiHeadAttentionEfficient* del *NoteBook transformer.py*:

```
class MultiQueryAttention(nn.Module):
    def __init__(self, n_embd, n_head, attn_pdrop=0.1, resid_pdrop=0.1):
        super().__init__()
        assert n_embd % n_head == 0

        # el tamaño de salida es n_embd (para Q) + 2 * head_dim (para K
        y V compartidos)
        self.c_attn = nn.Linear(n_embd, n_embd + 2 * self.head_dim)
        self.c_proj = nn.Linear(n_embd, n_embd)
        self.attn_dropout = nn.Dropout(attn_pdrop)
        self.resid_dropout = nn.Dropout(resid_pdrop)
        self.n_head = n_head
        self.n_embd = n_embd
        self.head_dim = n_embd // n_head # C'

    def forward(self, x, mask=None):
        B, T, C = x.size()
        H = self.n_head
        Cp = self.head_dim

        # dividimos el resultado de forma asimétrica
        # Q se lleva 'C' dimensiones, K y V se llevan 'Cp' dimensiones
        cada uno
        q, k, v = self.c_attn(x).split([C, Cp, Cp], dim=2)
        q = q.view(B, T, H, Cp).transpose(1, 2)
        # reformateamos K y V para que tengan 1 sola cabeza (H=1) en esa
        dimensión
        k = k.view(B, T, 1, Cp).transpose(1, 2) # (B, 1, T, Cp)
        v = v.view(B, T, 1, Cp).transpose(1, 2) # (B, 1, T, Cp)

        mask = mask.view(1, 1, T, T)

        # broadcasting aquí de 1 a H
        att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(Cp))
        att.masked_fill_(mask, float('-inf'))
        att = F.softmax(att, dim=-1)
        att = self.attn_dropout(att)
        # broadcasting de nuevo al multiplicar por 'v'
        y = att @ v
```

```
y = y.transpose(1, 2).contiguous().view(B, T, C)
y = self.resid_dropout(self.c_proj(y))
return y
```

Listing 9: Implementación de la función *Multi-Query attention*.

A nivel de implementación, la transición de **Multi-Head Attention** (MHA) a **Multi-Query Attention** (MQA) requiere cuatro modificaciones clave en el código:

- **Proyección lineal optimizada:** En la versión clásica, se proyectan las consultas (Q), claves (K) y valores (V) multiplicando la dimensión total por tres ($3 \times n_embd$). En MQA, la capa lineal proyecta Q con el tamaño original, pero reduce el espacio para K y V al tamaño de una única cabeza ($n_embd + 2 \times head_dim$).
- **División asimétrica:** El tensor resultante de la proyección ya no se divide en tres bloques idénticos. En su lugar, se le pasa a PyTorch una lista de tamaños exactos (`[C, Cp, Cp]`) para extraer Q con su dimensión completa y K y V con sus dimensiones reducidas.
- **Redimensionar a una sola cabeza:** Al aplicar la función `view` para preparar los tensores, Q mantiene su división en múltiples cabezas (H). Sin embargo, a K y V se les asigna explícitamente una única cabeza ($H = 1$).
- **Aprovechamiento del *broadcasting*:** En los pasos finales, al calcular el producto punto entre Q y K, y al multiplicar por V, las dimensiones incompatibles se resuelven mediante el motor de *broadcasting* de PyTorch que expande virtualmente la cabeza única de K y V desde 1 hasta H , realizando el cálculo para todas las consultas sin duplicar los datos reales en memoria.

Para reducir el tiempo de ejecución en la generación de texto, se utiliza una técnica llamada Caché KV. En lugar de recalcular desde cero las matrices de Claves (K) y Valores (V) en cada nuevo paso, estas se almacenan temporalmente en la memoria, ahorrando una gran cantidad de cálculos repetitivos.

Sin embargo, esta caché tiene un problema fundamental ya que requiere guardar una enorme cantidad de datos, pues en la *Multi-Head Attention* se debe mantener una K y una V distintas por cada cabeza.

Aquí es donde destaca la *Multi-Query Attention* (MQA) vista en el código anterior. La idea principal de MQA es mantener múltiples cabezas independientes para las consultas (*queries*), pero obligarlas a compartir una única matriz de Claves y otra de Valores. De

esta manera, el tamaño de la caché KV y el coste de memoria se reducen, a costa de una ligera disminución en la capacidad expresiva del modelo.

A continuación, implementamos una caché KV en nuestra función para aprovechar estas ventajas:

```
class MultiQueryAttentionCacheKV(nn.Module):
    def __init__(self, n_embd, n_head, attn_pdrop=0.1, resid_pdrop=0.1):
        super().__init__()
        assert n_embd % n_head == 0
        self.c_attn = nn.Linear(n_embd, n_embd + 2 * self.head_dim)
        self.c_proj = nn.Linear(n_embd, n_embd)
        self.attn_dropout = nn.Dropout(attn_pdrop)
        self.resid_dropout = nn.Dropout(resid_pdrop)
        self.n_head = n_head
        self.n_embd = n_embd
        self.head_dim = n_embd // n_head  # C'

        # k cache y v cache
        self.k_cache = None
        self.v_cache = None

    # funcion para limpiar cache
    def limpiar_cache(self):
        self.k_cache = None
        self.v_cache = None

    def forward(self, x, mask=None):
        B, T_current, C = x.size()
        H = self.n_head
        Cp = self.head_dim

        # si entran varios tokens a la vez o si cambia el tamaño del
        batch,
        # reseteamos la memoria
        if self.k_cache is not None and (T_current > 1 or self.k_cache.
size(0) != B):
            self.limpiar_cache()

        q, k, v = self.c_attn(x).split([C, Cp, Cp], dim=2)
        q = q.view(B, T_current, H, Cp).transpose(1, 2)
        k = k.view(B, T_current, 1, Cp).transpose(1, 2)
        v = v.view(B, T_current, 1, Cp).transpose(1, 2)
```

```

if self.k_cache is None or self.v_cache is None:
    self.k_cache = k
    self.v_cache = v
else:
    self.k_cache = torch.cat([self.k_cache, k], dim=2)
    self.v_cache = torch.cat([self.v_cache, v], dim=2)

# T_total es el tamaño de memoria interna acumulada
T_total = self.k_cache.size(2)

mask = mask.view(1, 1, T_current, T_total)

if mask is not None:
    # solo aplicamos la máscara si se procesa la frase entera (
entrenamiento)
    if T_current == T_total:
        mask = mask.view(1, 1, T_current, T_total)
        att.masked_fill_(mask, float('-inf'))

att = F.softmax(att, dim=-1)
att = self.attn_dropout(att)

y = att @ self.v_cache

y = y.transpose(1, 2).contiguous().view(B, T_current, C)
y = self.resid_dropout(self.c_proj(y))

return y

```

Listing 10: Implementación de una caché KV, a nuestra función *Multi-Query attention*.

Las modificaciones introducidas en esta función han sido las siguientes:

- **Caché:** Se inicializan los atributos `self.k_cache` y `self.v_cache` en el constructor para retener la información entre pasos sucesivos, junto con una función auxiliar `limpiar_cache()` para reiniciar el estado cuando sea necesario.
- **Auto-limpieza:** Se añade una validación lógica al inicio del paso hacia adelante (*forward pass*). Si el modelo recibe múltiples tokens simultáneos (`T_current > 1`) o cambia el tamaño del lote, deduce que está en fase de entrenamiento o procesando un texto inicial, y vacía la caché automáticamente para evitar colisiones de memoria.
- **Concatenación:** Cuando la caché está activa, los tensores K y V del nuevo token

no reemplazan a los anteriores, sino que se concatenan (`torch.cat`) a lo largo de la dimensión temporal (`dim=2`). Esto permite que la memoria `T_total` crezca en cada paso.

- **Máscara:** La máscara solo se activa si el modelo procesa toda la secuencia de golpe (`T_current == T_total`). Durante la generación de texto (un token a la vez), la máscara se ignora permitiendo que el token actual consulte todo el historial almacenado en la caché.

5.2. Estudio

Vamos a realizar un pequeño estudio sobre como afecta el uso de *multi-query attention* en el modelo. Hemos tomado 9 frases, la primera es la que se plantea en el *notebook*, después hemos elegido dos frases famosas de Shakespeare, pero que no están en el texto. Las siguientes 3 frases son del texto, y las 3 últimas hemos modificado alguna palabra. Para medir las probabilidades hemos usado la siguiente función:

```
def evaluar_probabilidad_frase(model, frase, word_index, device):
    """
    Calcula la probabilidad logarítmica promedio que el modelo
    le asigna a una frase específica.
    """
    model.eval()

    # convertir la frase a IDs
    words = frase.split()
    if len(words) < 2:
        return 0.0 # se necesita al menos contexto y una palabra a
    predecir

    # se traduce cada palabra a su ID usando el diccionario, si
    encuentra una
    # palabra que el modelo nunca ha visto, le asigna el ID de [UNK]
    input_ids = [word_index.get(word, word_index.get('[UNK]', 0)) for
    word in words]

    # Convertimos a tensor (1, T)
    input_tensor = torch.LongTensor(input_ids).view(1, -1).to(device)

    with torch.no_grad():
```

```

# pasamos toda la frase por el modelo a la vez
logits = model(input_tensor)

# convertimos los logits a probabilidades usando softmax
probs = F.softmax(logits, dim=-1)

log_prob_acumulada = 0.0

# iteramos para ver qué probabilidad le dio a la palabra
correcta en cada paso
# iteramos hasta el penúltimo token, porque el último token
# predice una palabra que ya no está en nuestra frase.
for i in range(len(input_ids) - 1):
    # i es el contexto actual. i+1 es la palabra real que
    queremos que haya predicho
    id_palabra_real_siguiente = input_ids[i + 1]

    # extraemos la probabilidad que el modelo le asignó a esa
    palabra real
    probabilidad = probs[0, i, id_palabra_real_siguiente].item()

    # sumamos el logaritmo (sumamos 1e-10 para evitar hacer log
    de 0 y que explote)
    log_prob_acumulada += math.log(probabilidad + 1e-10)

# promedio para poder comparar frases largas con frases cortas
score_promedio = log_prob_acumulada / (len(input_ids) - 1)

# transformación a probabilidad media real (ej: 0.10 -> 10%)
probabilidad_media_real = math.exp(score_promedio)

# transformación a perplejidad
perplejidad = math.exp(-score_promedio)

print(f"Probabilidad media por palabra: {probabilidad_media_real *
100:.2f}%")
print(f"Perplejidad del modelo: {perplejidad:.2f}")

return score_promedio

```

Listing 11: Implementación de una función para medir las probabilidades.

El estudio se ha realizado tanto para MultiHeadAttention (MHA), como para Multi-QueryAttention (MQA) y se pueden ver los resultados en la Tabla 5

Texto	Prob. media (%)	Perplejidad	Score
O God, that robot is out of control! I tell you, friends (MHA / MQA)	0.07 / 0.11	1483.50 / 933.31	-7.3022 / -6.8387
Doubt that the stars are fire, doubt that the sun moves... (MHA / MQA)	0.01 / 0.01	12169.19 / 15650.80	-9.4067 / -9.6583
To be, or not to be, that is the question (MHA / MQA)	0.01 / 0.05	13259.64 / 1848.44	-9.4925 / -7.5221
Before we proceed any further, hear me speak. (MHA / MQA)	56.93 / 51.45	1.76 / 1.94	-0.5634 / -0.6645
You are all resolved rather to die than to famish? (MHA / MQA)	38.65 / 56.79	2.59 / 1.76	-0.9507 / -0.5657
First, you know Caius Marcius is chief enemy to the people. (MHA / MQA)	52.48 / 67.86	1.91 / 1.47	-0.6447 / -0.3877
Let me kill you, and we will have corn at our new price. (MHA / MQA)	0.49 / 0.29	203.09 / 341.42	-5.3136 / -5.8331
Two words, good citizens. (MHA / MQA)	0.22 / 0.66	446.50 / 152.40	-6.1014 / -5.0265
With him last: he is a very dog to the commonalty. (MHA / MQA)	2.43 / 1.24	41.15 / 80.84	-3.7171 / -4.3925

Tabla 5: Comparación entre Multi-Head Attention (MHA) y Multi-Query Attention (MQA) en las mismas muestras de texto.

Vemos que MHA y MQA alcanzan un rendimiento similar, a lo largo de todas las frases. Primero, la probabilidad que presenta la frase planteada en el notebook es muy baja en ambos modelos. Por otro lado, obtenemos unas probabilidades incluso más bajas con frases famosas de Shakespeare, esto posiblemente debido a que los modelos no se han entrenado con suficientes datos para mejorar la capacidad de generalizar.

Sin embargo, un escenario completamente distinto nos encontramos con las frases del conjunto de entrenamiento, vemos cómo sus probabilidades se encuentran en un rango aceptable [35, 70], además de una perplejidad consistente, donde la media más alta es 2.59, es decir, el modelo considera menos de 3 opciones efectivas, por lo general.

Por último, en las frases que han sido modificadas, vemos un punto medio entre los casos anteriores. Con un rango entre [0.20, 2.50], vemos como aún estamos lejos de considerarlo aceptable. Fijándonos en la perplejidad, vemos como el mejor caso, alcanza más de 40 opciones efectivas, lo cual es una estimación ambigua.

Vemos unos resultados bastante limitados debido posiblemente a la falta de datos o la simplicidad de nuestros modelos.

Otro aspecto fundamental en nuestra evaluación fue la eficiencia computacional, medida a través del tiempo de ejecución de cada modelo. Los tiempos registrados se muestran en la Tabla 6:

	Tiempo (Segundos)
MultiHeadAttention simple	22.10 segundos
MultiQueryAttention	20.47 segundos
MultiQueryAttention con Caché KV	20.07 segundos

Tabla 6: *Tabla vacía 3x3*

Teóricamente, puesto que Multi-Query Attention (MQA) obliga a todas las cabezas a compartir una única matriz de claves (K) y valores (V), reduce tanto el número de operaciones como la cantidad de datos en memoria. Por ello, es esperable observar un menor tiempo de ejecución en comparación con la atención clásica. Esta eficiencia debería ser aún más pronunciada al incorporar la caché KV, ya que evita recalculiar el contexto de los tokens pasados durante la generación.

Sin embargo, al observar los resultados (22.10 s, 20.47 s y 20.07 s), la mejora de rendimiento es ínfima. Debemos tener en cuenta que estamos trabajando con modelos de tamaño muy reducido donde el beneficio de MQA y la caché KV no llegan apenas a apreciarse.

Aunque para realizar un estudio concluyente necesitaríamos incrementar el tamaño del vocabulario, los parámetros de la red y la longitud de las secuencias, estos resultados confirman la tendencia esperada.